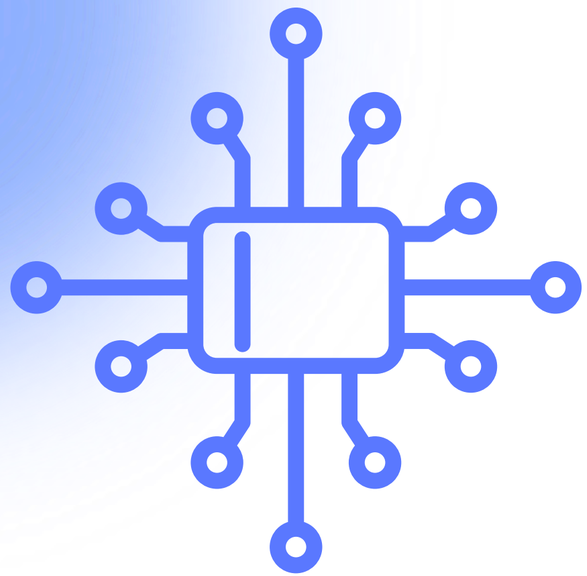


TRANSFER LEARNING

MADE BY:

SANDY ALAA

INTRODUCTION

Humans never learn from zero. We reuse what we already know:

Learn a Task → Keep the Knowledge → Reuse it on a New Task

Transfer Learning: is a technique where a model trained on one task (the source task) is reused as the starting point for a different but related task (the target task). Instead of learning everything again, the model transfers the knowledge it already has.

Example: someone who can ride a bicycle learns to ride a motorcycle much faster, because balance and steering are already learned.



WHY TRANSFER LEARNING?

Training a deep network from zero needs millions of labeled images, powerful GPUs and a lot of time.
Transfer learning lets us skip most of that cost.

Less Data
Works well with only a few hundred images per class.

Faster Training
Minutes or hours instead of days or weeks.

Less Computing Power
A single GPU or a free Colab session is often enough.

Better Accuracy
Starts from strong general features, so results are usually higher.

Less Overfitting
Pretrained weights generalize better than random ones on small data.

Reused Knowledge
Builds on what large teams trained on millions of images.

TRAINING FROM SCRATCH

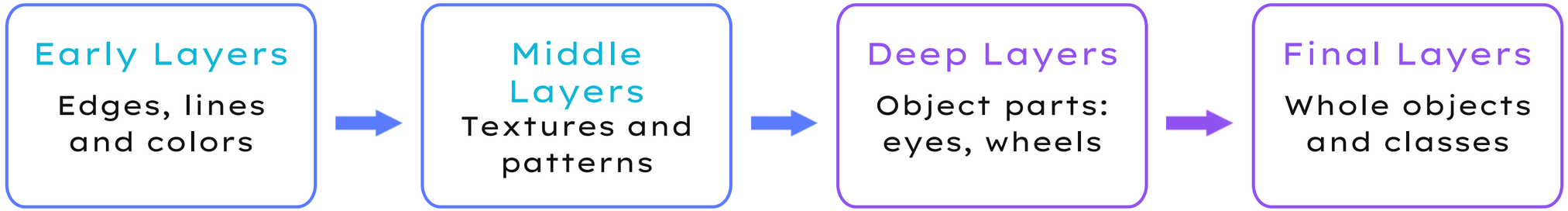
Training from Scratch: all weights start as random numbers, and the network must learn everything (edges, shapes, objects) only from our own dataset.

Aspect	From Scratch	Transfer Learning
Starting weights	Random	Pretrained (e.g. on ImageNet)
Data needed	Very large (100K+ images)	Small to medium
Training time	Days to weeks	Minutes to hours
Hardware	Powerful GPUs	Single GPU or Colab
Overfitting risk	High with small data	Lower
When to use	Huge dataset or a very different domain	Most real-world projects

PRETRAINED MODELS

Pretrained Model: is a network that has already been trained on a large dataset. Its learned weights are saved and shared, ready to be reused. Most vision models are pretrained on ImageNet (about 1.2 million images in 1,000 classes).

What does a CNN learn at each depth?



Early features are general and useful for almost any image task.
Late features are specific to the original task, so they are the ones we usually replace or retrain.

TWO WAYS TO REUSE

A pretrained CNN has two parts: the backbone (feature layers) and the head (final classifier).
We always replace the head to match our classes. Then we choose:

Feature Extraction

Use the backbone as a fixed feature extractor: its weights are not changed. Only the new classifier head is trained on our data.

- Fast and cheap
- Works with very little data
- Lower risk of overfitting

Fine-Tuning

Start from the pretrained weights and continue training some or all layers with a small learning rate, so the features adapt to the new task.

- Usually higher accuracy
- Needs more data and time
- Must be careful not to overfit

FAMOUS CNN ARCHITECTURES

Each of these models introduced an idea that changed how CNNs are designed,
and all of them are available today as pretrained models.

2012	2014	2014	2015	2017	2017	2019
AlexNet	VGG	Inception	ResNet	DenseNet	MobileNet	EfficientNet
ReLU, Dropout and GPU training	Simple stacks of 3×3 convolutions	Parallel filters of different sizes	Skip connections, up to 152 layers	Each layer linked to all later layers	Light and fast for mobile devices	Balanced scaling of depth, width and resolution

The trend: deeper → smarter connections → more efficient



ALEXNET & VGG

AlexNet

2012 · Krizhevsky, Sutskever & Hinton

- Won the ImageNet 2012 challenge by a huge margin (top-5 error ≈15.3% vs 26.2%) and started the deep learning era
- 8 learned layers: 5 convolutional + 3 fully connected
- About 60 million parameters
- Popularized ReLU, Dropout, data augmentation and training on GPUs
- Uses large filters (11×11 and 5×5)

VGG

2014 · Visual Geometry Group, Oxford

- Very simple, uniform design: only 3×3 convolutions and max pooling
- Two stacked 3×3 layers see the same area as one 5×5, with fewer parameters and more non-linearity
- Versions: VGG16 and VGG19 (number of layers)
- About 138 million parameters (VGG16): heavy and slow
- Still popular for feature extraction and style transfer

INCEPTION & DENSENET

Inception (GoogLeNet)

2014 · Google

- Won ImageNet 2014 with a 22-layer network
- Inception module: runs 1×1, 3×3 and 5×5 convolutions and pooling in parallel, then joins the results
- The network learns which filter size is useful
- 1×1 convolutions shrink the channels, making it cheap to compute
- Only ~6.8M parameters; later versions: Inception v3, v4

DenseNet

2017 · Huang et al.

- Each layer receives the feature maps of all previous layers in its block (dense connections)
- Features are concatenated (joined), not added as in ResNet
- Encourages feature reuse, so it needs fewer parameters
- Strong gradient flow makes training easier
- DenseNet-121: ~8M parameters; downside: high memory use

MOBILENET & EFFICIENTNET

MobileNet

2017 · Google

- Built for phones and embedded devices
- Depthwise separable convolution: one filter per channel, then a 1×1 convolution to mix channels, about 8 to 9× less computation
- Width and resolution multipliers trade accuracy for speed
- V2 adds inverted residual blocks; V3 was found by architecture search
- MobileNetV2: only ~3.5M parameters

EfficientNet

2019 · Google

- Compound scaling: grows depth, width and image resolution together with a fixed ratio, instead of only one of them
- Base model B0 was designed by neural architecture search
- Family B0 → B7: bigger means more accurate but slower
- B0: ~5.3M parameters, ~77% top-1; B7: ~66M, ~84% top-1
- Excellent accuracy for its size

VANISHING GRADIENT

Before ResNet, people expected that deeper networks = better results.
But simply stacking more layers caused two problems:

Vanishing Gradient

During backpropagation the gradient is multiplied by small numbers at every layer. After many layers it becomes almost zero, so the early layers stop learning.

Degradation Problem

A 56-layer plain network had higher training error than a 20-layer one. It was not overfitting: the deeper model was simply harder to optimize.

The gradient flows backward and shrinks (example: ×0.5 at each layer)

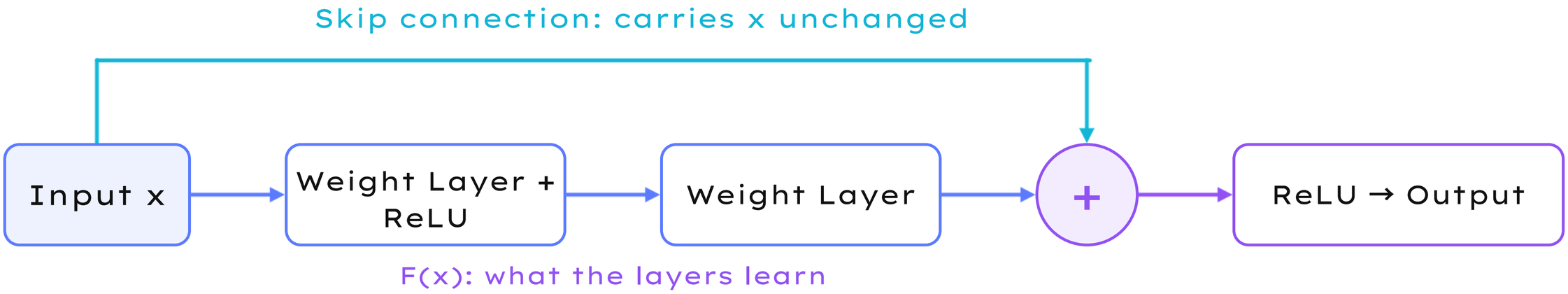
Layer 1	Layer 2	Layer 3	Layer 4	Layer 5	Output
0.03	0.06	0.13	0.25	0.50	1.00

The solution: ResNet and its skip connections

RESIDUAL BLOCKS

ResNet (Residual Network): introduced by Microsoft Research in 2015 (Kaiming He et al.). It won ImageNet 2015 with networks up to 152 layers deep.

Residual Block: instead of learning the full output, a few layers learn only the residual F(x): the difference that must be added to the input.



$$\text{Output} = F(x) + x$$

SKIP CONNECTIONS

Skip Connection (Shortcut): a path that jumps over one or more layers and adds the input directly to the output of those layers. It is what makes very deep networks trainable.

Identity Shortcut

Used when input and output have the same shape. The input x is added as it is, with no extra parameters.

Projection Shortcut

Used when the size or number of channels changes. A 1×1 convolution reshapes x so it can be added.

Gradient Highway

The gradient of F(x) + x always contains a +1 term, so it can flow back through the shortcut without vanishing.

Result: networks with 50, 101 and even 152 layers train without degradation

RESNET18 VS RESNET50

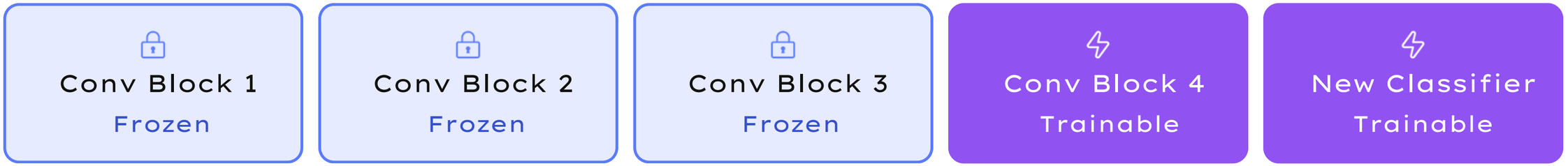
The ResNet family comes in depths of 18, 34, 50, 101 and 152 layers. The number is the count of layers with weights.

Feature	ResNet18	ResNet50
Block type	Basic block: two 3×3 convs	Bottleneck: 1×1 → 3×3 → 1×1 convs
Blocks per stage	2 – 2 – 2 – 2	3 – 4 – 6 – 3
Parameters	~11.7 million	~25.6 million
ImageNet top-1	~69.8%	~76.1%
File size	~45 MB	~98 MB
Speed	Fast and light	Slower, more accurate
Best for	Small datasets, limited hardware, quick tests	Higher accuracy; the most common default backbone

Bottleneck block: the first 1×1 conv shrinks the channels, the 3×3 conv processes them, and the last 1×1 conv expands them again. This lets ResNet50 go deeper at a low cost.

FREEZING LAYERS

Freezing: locking a layer's weights so they are not updated during training. The layer still processes the data (forward pass), but the optimizer never changes it.



Frozen Layers

- Usually the early layers (general features)
- Keep the knowledge learned on ImageNet
- Faster training and less GPU memory

Trainable Layers

- Usually the last layers and the new head
- Weights are updated by backpropagation
- Adapt the model to the new task

FINE-TUNING STRATEGIES

Fine-Tuning: continuing to train a pretrained model on new data so it adapts to the new task.

It can be done at three levels:

Feature Extraction



Freeze the whole backbone; train only the new head.

- Data: small
- Speed: fastest, least memory
- Lowest overfitting risk

Partial Fine-Tuning



Freeze early layers; train the last blocks + head.

- Data: medium
- Speed: medium
- Adapts high-level features, keeps general ones

Full Fine-Tuning



Unfreeze all layers; pretrained weights are only the start.

- Data: large
- Speed: slowest, most memory
- A high learning rate can erase useful features

Frozen layer

Trainable layer

WHICH STRATEGY TO USE?

	Similar to ImageNet	Different domain
Small dataset	**Feature Extraction** e.g. cats vs dogs. Train only the new head.	**Partial Fine-Tuning** e.g. few X-ray images. Unfreeze a few top blocks and use strong augmentation.
Large dataset	**Partial or Full Fine-Tuning** Enough data to adapt more layers safely.	**Full Fine-Tuning** Or training from scratch if the data is huge and very different.

Golden tips:

- Use a small learning rate (often 10× smaller) so pretrained weights change gently
- Train the new head first, then unfreeze gradually from the top layers down
- Prepare images the same way as the original training (e.g. 224×224, ImageNet normalization)

CHOOSING A MODEL

There is no single best pretrained model. The right choice depends on four factors:

Accuracy
How well it performs, often measured by ImageNet top-1 accuracy. Matters most when mistakes are costly (medical, security).

Speed
Time per image (latency) and frames per second. Real-time apps like cameras and robots need fast models.

Model Size
Number of parameters and file size in MB. Important for mobile apps, downloads and limited memory.

Hardware Requirements
GPU memory, RAM, CPU or edge device. Big models need strong GPUs; phones need tiny models.

Always a trade-off: higher accuracy ↔ bigger size, slower speed, stronger hardware

THANK YOU

I hope you understand the session well

Thanks for your time

CONTACT US:

Sandy's LinkedIn

+20 1006336118